Retail Data Warehouse Project Report

MESHJOIN and OLAP analysis

Abdullah Bukhari
Student ID 24I-2601 Section DS-5A
Instructor Dr. Asif Naeem
DS3003 and DS3004 Fall 2026
13 November 2026

1 Project overview

This project builds a retail data warehouse from the UCI Online Retail workbook and processes two prescribed transaction channels through a disk-based MESHJOIN implementation. OnlineThread generates paced live input, PeriodicThread releases ordered hourly groups, and MeshJoinThread enriches each line from customer and product master pages before warehouse loading. The resulting star supports the twenty required OLAP questions.

The completed required pipeline reconciled every retained source line and all dimension attributes in MySQL 8.0.45. Both channel totals matched the cleaned source exactly. All twenty analytical queries passed independent source-based comparisons, and 51 regression tests passed. These outcomes verify the delivered dataset and tested recovery paths. Accelerated demonstrations do not establish production throughput.

Verified measure	Result
Retained transaction facts	397,884
Quantity sold	5,167,812 units
Stored revenue	GBP 8,911,407.904000
Rows across five dimensions	25,295
Required OLAP answers	20 of 20 passed

Scope of the delivered loading mode

Online results enter fact_sales directly after consolidation. Periodic results are prepared durably and published in hourly batches. The optional online cache bonus is not implemented in this report version. The report describes the verified required implementation and makes no bonus claim. The supplied project brief is the requirements authority [1].

2 Dataset preparation

The workbook contains 541,909 data rows with InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, and Country. These map to invoice, product, description, quantity, event time, price, customer, and country fields. Identifiers are preserved as text, prices use Decimal arithmetic, and historical event times remain timezone-naive. Coverage is 1 December 2010 at 08:26 through 9 December 2011 at 12:50.

Programmatic cleaning removes an invoice beginning with C, a missing customer ID, quantity at or below zero, or price at or below zero. It prints before-and-after counts and leaves the workbook unchanged.

Measure	Rows
Original input	541,909
Retained after cleaning	397,884
Removed by union of exclusions	144,025
Cancellation condition	9,288
Nonpositive quantity condition	10,624
Nonpositive price condition	2,517
Missing customer condition	135,080

Exclusion conditions overlap, so their counts cannot be summed to calculate removals. The reconciliation is 397,884 retained plus 144,025 removed equals 541,909 input rows. The raw workbook contains 5,268 exact repeats of previously seen eight-column rows. Valid repeated lines are retained individually under the required transaction grain.

Each original workbook line receives source_line_id before filtering. This identity distinguishes repeated invoice/product lines and permits safe retries, while transaction_id remains available for shopping-cart analysis. Description quality is handled during product-master generation rather than by removing otherwise valid sales.

Provenance and acquisition limitation

The workbook was downloaded on 12 November 2026. A complete public-mirror copy was used after slow UCI transfers. The 2,134,016-byte prefix obtained directly from UCI matched it exactly; ZIP integrity, published row count, columns, and date coverage passed inspection. Full byte equality against a complete UCI download was not established. Data/data_notes.md records the unchanged-file checksum, source and mirror URLs, and required attribution [2].

3 Channel assignment and master preparation

Let h = int(MD5(transaction_id encoded as UTF-8).hexdigest()[:8], 16). An invoice is assigned Online if h modulo 10 is 0 to 6, and Periodic if it is 7 to 9. Every line in that invoice retains the same channel. Online uses store 1; Periodic uses store 2 + (h modulo 4). The channels and stores are assignment-generated rather than observed UCI channel attributes.

Store ID	Store name	City	Assigned lines
1	Online	Web	278,740
2	London Bridge	London	21,053
3	Manchester Arndale	Manchester	37,302
4	Birmingham Bullring	Birmingham	19,850
5	Edinburgh Princes	Edinburgh	40,939

Deterministic master attributes

The cleaned input yields 4,338 customer and 3,665 product master rows. stable(text, n) takes the same eight-digit MD5 prefix modulo n. Customer gender, age, occupation, city category, years in city, and marital status follow the exact suffixes and ordered lists in the brief. City category uses the first cleaned country for that customer. These generated demographics must not be interpreted as measured customer characteristics.

Product category and supplier follow the prescribed hash rules. Product name is the longest retained description, with the first occurrence retained on equal-length ties and UNKNOWN used when the selected description is empty. The supplier definitions and five in-memory stores are fixed by the assignment. Every generated attribute was independently checked.

Partitions and hourly groups

At 500 master tuples per page, the measured partition counts are ceiling(4,338 / 500) = 9 for customers and ceiling(3,665 / 500) = 8 for products. Short final pages are included. MESHJOIN does not load either full master into a Python lookup dictionary.

The 119,144 Periodic lines are sorted by event_time and source_line_id, then divided into 48 contiguous balanced groups. Groups 0 to 7 have 2,483 rows each; groups 8 to 47 have 2,482. Staging records release_hour and released. Group 0 may start immediately, and later extractions remain at least one configured hour apart. The normal simulated hour is 60 real seconds.

4 Star schema and transaction grain

One fact represents one retained product line in one shopping cart, with one customer, supplier, store, and event time. sales_key is the surrogate primary key; source_line_id is unique. Each dimension retains its original business ID and has a surrogate key referenced by a non-null fact foreign key.

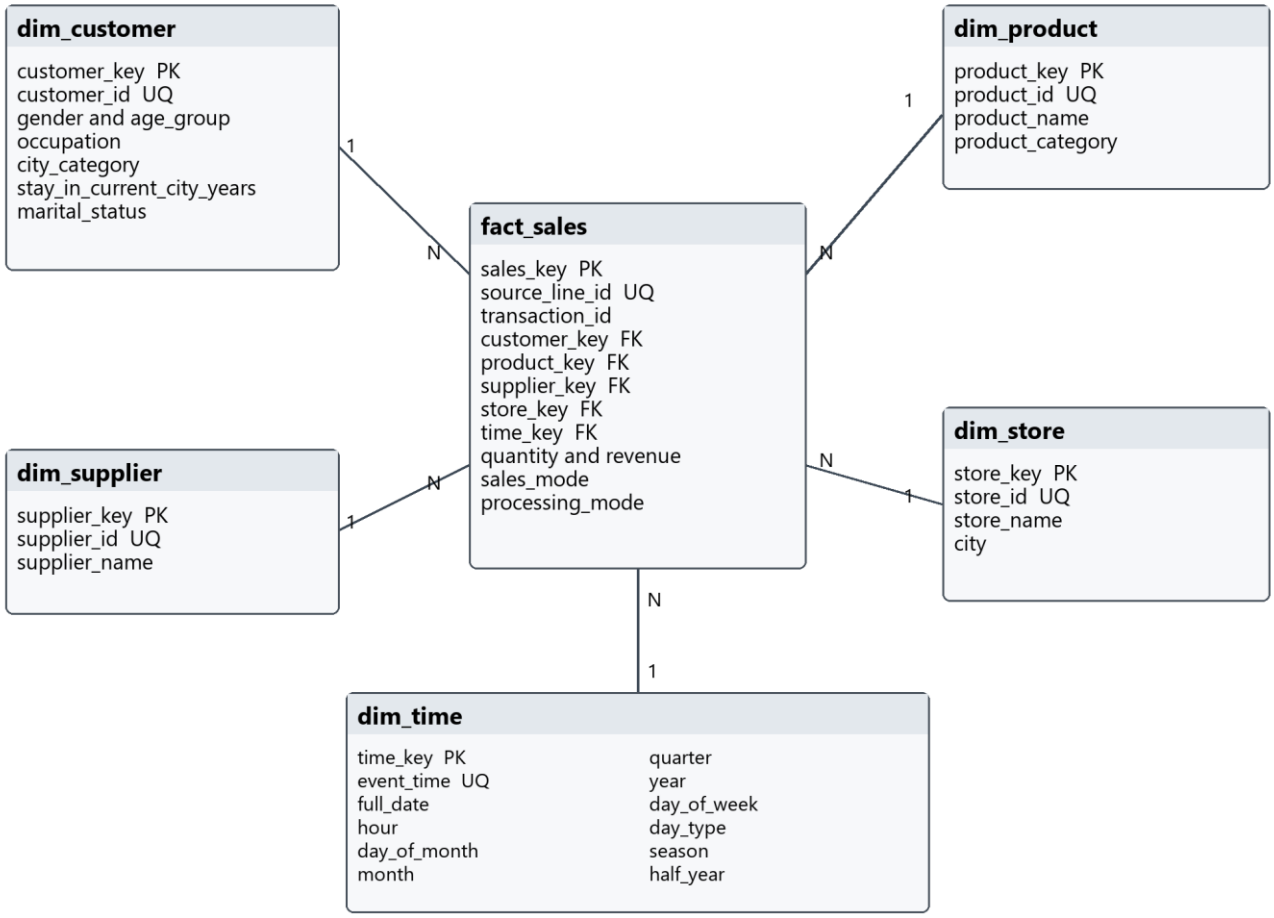


Figure 1 Implemented star schema. Each dimension has a one-to-many relationship with fact_sales. PK means primary key, FK foreign key, and UQ unique key. Working ETL tables are outside the star.

fact_sales stores quantity, exact DECIMAL(24,6) revenue, transaction_id, sales_mode, and processing_mode. Revenue is calculated after both master matches as quantity multiplied by unit price. No cart-level aggregation replaces line facts, and report totals are not stored as a second business fact.

dim_time stores event time, date, hour, day of month, month, quarter, year, ISO weekday number, day type, season, and half-year. Monday to Friday are weekdays. Spring is March to May, summer June to August, fall September to November, and winter December to February. The first half is January to June. There are 17,282 distinct loaded event times.

5 MESHJOIN and independent execution

MESHJOIN cyclically scans customer and product masters kept on disk, reading one page of at most 500 tuples from each per active iteration. Its shared stream buffer holds at most 1,000 transactions. H is a multimap indexed by master kind and customer/product ID. Buckets and the chunk deque reference the same pending objects, so repeated keys never overwrite separate sales.

1. Read the next page of each master into separate disk buffers. Page readers wrap after the actual 9 and 8 partitions.
2. Before admission, eligible peak overload may move cleaned Periodic buffer rows to disk. Admit up to w = 500 stream rows as a chunk, and index each pending object under both master keys.
3. Probe H with each page tuple and retain matching customer/product attributes. Producers can continue filling the bounded shared buffer while the join scans.
4. Age each chunk once per iteration. Retain it for max(9, 8) = 9 iterations so both cyclic master scans cover every partition, regardless of its admission position.
5. When the oldest chunk has completed coverage, require both matches for every member. Deliver the complete enriched chunk synchronously. Remove its H entries, deque references, and input ownership only after successful downstream acceptance.

A missing master match raises an error after full coverage. A failed output callback retains the chunk for retry before further scanning. Partial-read or input failure stops the run rather than claiming coverage succeeded. The warehouse callback supplies the durable, idempotent handoff. A complete core fixture enriched 397,884 rows in 804 iterations, reading 804 customer and 804 product pages; this is separate from warehouse timing.

Thread ownership and priorities

OnlineThread reads the seven-column CSV and matching assigned input to recover source-line identity. PeriodicThread owns its staging connection, extracts one group, and waits for downstream acknowledgements. MeshJoinThread owns the master reader and separate warehouse-writer and shed connections. Connections are created, used, and closed by their owning workers. The buffer prioritizes Online, then fresh Periodic, then shed replay, with stable ordering inside each class.

Operator control

The console accepts online, periodic, both, peak, non-peak, status, and quit. Optional positive rates after the speed commands update them at runtime. online pauses future periodic extraction; periodic pauses the stream reader; both enables both channels. Accepted work and an already extracted group continue draining. Status reports mode, speed/rates, stream count, H transactions, queue chunks, shed count, and progress. Defaults are peak 80/second and non-peak 10/second.

6 Durable loading and recovery

Shedding and replay

Only cleaned Periodic transactions are shed, only during peak overload when the shared buffer is actually full. A database commit precedes removal from memory. Online rows and replay entries are not selected. In non-peak mode, available capacity admits eligible disk rows for the current group; replay retains original IDs, quantity, price, event time, and modes. Disk ownership is removed only after durable downstream acceptance. Restart recognizes matching shed IDs instead of submitting the same staging rows twice.

Consolidation and warehouse publication

Both master matches precede consolidation. The in-memory store list supplies store attributes, event time supplies calendar parts, and Decimal arithmetic supplies revenue. Dimension business keys are resolved or inserted before fact foreign keys are written. Online processing_mode defaults to Live; staged processing_mode remains Periodic, including after shedding.

Online results load directly to facts. Joined Periodic results persist to periodic_join_results, a durable ETL handoff outside the star. Once a whole group is acknowledged, its facts, released flags, and prepared-row removal commit together. Identical existing facts are verified and reused; conflicting attributes fail. Quit and Ctrl+C stop input and drain accepted work. Incomplete staging and prepared results remain available for retry. The optional Online cache bonus is not part of this implementation.

Measured MySQL runs

Run	Online and Periodic enriched	New shed	Replay	Seconds
Peak stress	10,000 and 2,483	1,785	1,785	1.249
Small warehouse load	100 and 2,483	1,000	1,000	3.028
Resumed full load	278,740 and 116,661	793	2,483	94.375

The stress check used a 1,000,000/second peak ceiling and default 60-second hour. The small load used peak 80 and a two-second hour. The resumed full load used a 1,000,000/second peak ceiling and one-second hour. Non-peak stayed at 10/second. These configured arrival ceilings are not measured throughput. Stress verification rolled back staging release flags; warehouse runs committed completed groups.

The full run inserted 89,033 Online and 116,661 Periodic facts and verified 189,707 existing Online facts. The first 2,483 Periodic facts were already committed. Replay included earlier persisted backlog, explaining why 2,483 rows replayed while 793 were newly shed. Its 94.375 seconds measure a resumed accelerated run, not a fresh full-dataset benchmark. All final buffers, H, queue, shed rows, and prepared results were zero.

7 Reconciliation and OLAP verification

Channel	Facts	Quantity	Stored revenue GBP
Online	278,740	3,526,585	6,075,186.073000
Periodic	119,144	1,641,227	2,836,221.831000
Total	397,884	5,167,812	8,911,407.904000

The initial small load produced 2,583 facts, 32,492 units, and GBP 63,009.850000. Its rerun inserted zero new facts. Native MySQL reconciliation then checked every source-line identity and fact field, all master/dimension attributes, calendar parts, per-channel totals, sorted group placement, released flags, and empty backlogs. All checks passed. The 51 regression tests include deliberate corruptions preserving aggregate totals, rollback, cancellation, replay, and retries.

Question	Analysis	Result rows	Check
1	Monthly product ranks	120	PASS
2	Customer purchase demographics	42	PASS
3	Category and occupation	100	PASS
4	Quarterly demographic trend	56	PASS
5	Top occupations per category	50	PASS
6	City and marital status by month	36	PASS
7	Purchase average by stay and gender	10	PASS
8	City ranks per category	30	PASS
9	Monthly category growth	120	PASS
10	Age and weekday weekend sales	14	PASS
11	Separate monthly product ranks	120	PASS
12	Quarterly store growth	20	PASS
13	Store supplier product name	15,010	PASS
14	Product sales by season	11,921	PASS
15	Store supplier monthly volatility	300	PASS
16	Top product pairs	5	PASS
17	Selected year ROLLUP	14,674	PASS
18	Product half and full year totals	3,596	PASS
19	High daily product revenue	114,713	PASS
20	Store quarterly view	25	PASS

Every native MySQL result row matched independently source-derived answers, ending PASS 20/20. Result sizes above count analytical rows, not source transactions. The selected year is 2011 through December 9; Q6 covers July to December 2011. Each query documents its scope, including partial-year and partial-month coverage.

8 Analytical conventions and revenue spikes

Q1 and Q11 separately rank five products within each month and weekday/weekend class, breaking revenue ties by product ID. Q5 ranks five occupations per category, with occupation text breaking ties. Q8 includes A, B, and C and uses equal dense ranks for equal revenue. Q7 averages cart amounts within gender/stay groups. Q16 counts unordered pairs after deduplicating products within each cart.

Q9, Q12, and Q15 include zero-sales periods and return NULL for first-period or zero-denominator percentage changes. Change is 100 times (current minus previous) divided by previous. Q17 rolls up business IDs to avoid redundant name subtotal levels. Q18 verifies that the two halves sum to the selected year [4]. Q20 is the regular view explicitly requested by Q20, ordered on reading by store name, year, and quarter; this precise instruction resolves the general materialized-view wording in section 11.

Daily product revenue spikes

Q19 uses all 374 calendar days from 1 December 2010 through 9 December 2011, including zero-sales dates. Its average is product revenue divided by 374. The strict unrounded test is daily_revenue multiplied by calendar_days greater than twice product_total. Display statistics are rounded after testing. It returned 114,713 qualifying product/date rows. The five highest daily revenue examples are below.

Product	Date	Daily GBP	Average GBP	Factor
23843	2011-12-09	168,469.60	450.453476	374.000000
23166	2011-01-18	77,183.60	217.691791	354.554480
22502	2011-06-10	39,916.50	126.647594	315.177722
85123A	2011-01-11	8,656.00	268.993316	32.179238
M	2011-10-27	8,322.12	143.796604	57.874246

The products are PAPER CRAFT , LITTLE BIRDIE; MEDIUM CERAMIC TOP STORAGE JAR; PICNIC BASKET WICKER 60 PIECES; WHITE HANGING HEART T-LIGHT HOLDER; and Manual, respectively. A factor of 374 means all product revenue lies on one day of the common window. Small one-day products also reach 374, such as a GBP 2.52 day for 17028J. Relative concentration therefore differs from absolute sales significance. Including pre-first-sale and post-last-sale zeros lowers averages for sparse products and helps explain the large number of flags.

Cleaning retains valid positive sales and removes cancellations/nonpositive lines, so revenue describes cleaned positive sales rather than net revenue after returns. December 2011 is partial and synthetic demographics/channels limit business generalization. Spikes identify concentration but do not establish a commercial cause or prove an erroneous sale.

9 MESHJOIN shortcomings and reflection

Repeated master scans

MESHJOIN rereads master pages even when transactions repeatedly reference a small subset of keys. Each active iteration here reads from both relations. Larger masters increase disk work and can reduce sustainable arrival capacity. This cost follows from the cyclic disk-scan design. Faster storage or a different indexed join strategy could reduce it, but the required implementation retains paged MESHJOIN behavior.

Retention latency and sensitivity to skew

A chunk waits for the larger master cycle even if some rows find both matches earlier. The coverage bound here is nine iterations. Admission position, relation size, and popular-key skew affect work and completion time. The 1,000-row stream limit bounds only input buffering; H, retained attributes, and chunk references also consume memory. More partitions extend retention and can increase pressure. Further optimization needs explicit memory and latency measurements.

Sustained overload and delayed periodic facts

A finite buffer cannot absorb a producer permanently faster than joining and loading. Shedding adds durable write/replay work and delays physical-store facts. Continued Online priority can lengthen Periodic waiting time, while replay depends on non-peak capacity and available disk. A production extension would need monitored backlog limits, capacity planning, and explicit fairness or backpressure policies. These are improvements to evaluate, not features claimed by the present tests.

Reflection

This project helped me distinguish the correctness of a final total from the correctness of each sale. Stable source-line IDs preserved valid repeats and made retries testable. Reconciliation showed why aggregate checks need identity, attribute, and calendar checks alongside them. I also learned that a correct join needs a durable handoff: a row cannot be forgotten merely because it entered a replay buffer.

The analyses made calculation choices equally important. Purchase averages need an explicit cart grain, pairs need distinct cart products, and growth needs defined zero baselines. Native MySQL initially combined Q13 names differing only in trailing spaces, producing 15,002 rather than 15,010 groups. Grouping HEX(product_name) and displaying the unchanged text fixed the issue without editing warehouse data. A PAD SPACE regression now reproduces it [3]. That comparison showed the value of an independent source calculation and native-engine verification.

Finally, recording defaults, accelerated settings, and resumed state kept timings and shed counts interpretable. Data preparation, concurrency, persistence, and analysis must be checked together. The successful final comparisons verify this dataset and the exercised recovery paths; they do not eliminate the algorithm's scaling limits.

10 Reproduction and references

The six required folders contain the schema, Python application, original/prepared data, analyses, report, and ReadMe. ReadMe/README.md provides complete setup and operation instructions. After environment setup, preparation, and complete loading, the principal checks are:

Check	Command
Database	python Mesh-Join/main.py check-db
Interactive processing	python Mesh-Join/main.py run
Every source line	python Mesh-Join/main.py reconcile
Twenty OLAP answers	python Mesh-Join/main.py check-queries
Regression suite	python -m unittest discover -s Mesh-Join/tests

Credentials are entered privately at runtime. Use run --batch for automatic finite execution. Rates and time-scale overrides print at startup. In Workbench, select the populated schema and choose Don't Limit before executing Queries-DW/analysis_queries.sql. The optional Online cache bonus would require separate cache fill/drain and fact-equivalence evidence before any bonus claim is added.

References

[1] DS3003 and DS3004 Practical Project. Fall 2026. Supplied DWH_Project.pdf, sections 5 to 13 and 16. Requirements for preparation, MESHJOIN, execution, warehouse loading, analyses, report, and attribution.

[2] Chen, D., Sain, S. L., and Guo, K. Online Retail. UCI Machine Learning Repository. Downloaded 12 November 2026.

https://archive.ics.uci.edu/dataset/352/online+retail

Acquisition mirror for the unchanged workbook:

https://raw.githubusercontent.com/pplonski/datasets-for-start/master/online-retail/Online-Retail.xlsx

[3] Oracle. MySQL 8.0 Reference Manual. The binary Collation Compared to _bin Collations. Trailing-space behavior used to resolve exact-name grouping.

https://dev.mysql.com/doc/refman/8.0/en/charset-binary-collations.html

[4] Oracle. MySQL 8.0 Reference Manual. GROUP BY Modifiers and Arithmetic Operators. ROLLUP grouping masks and decimal division semantics.

https://dev.mysql.com/doc/refman/8.0/en/group-by-modifiers.html

https://dev.mysql.com/doc/refman/8.0/en/arithmetic-functions.html

Implementation and evidence locations

Create-DW/create_dw.sql defines the schema. Mesh-Join/retail_dw contains the application and Mesh-Join/tests the regression fixtures. Data/data_notes.md records acquisition and preparation. Queries-DW/verification_notes.md records calculation conventions, row counts, Q13 correction, and Q19 examples. Project-Report/evidence/verification_record.md records authenticated run outcomes. Final native reconciliation and all twenty query comparisons passed, with OLAP verification confirmed on 13 November 2026.